



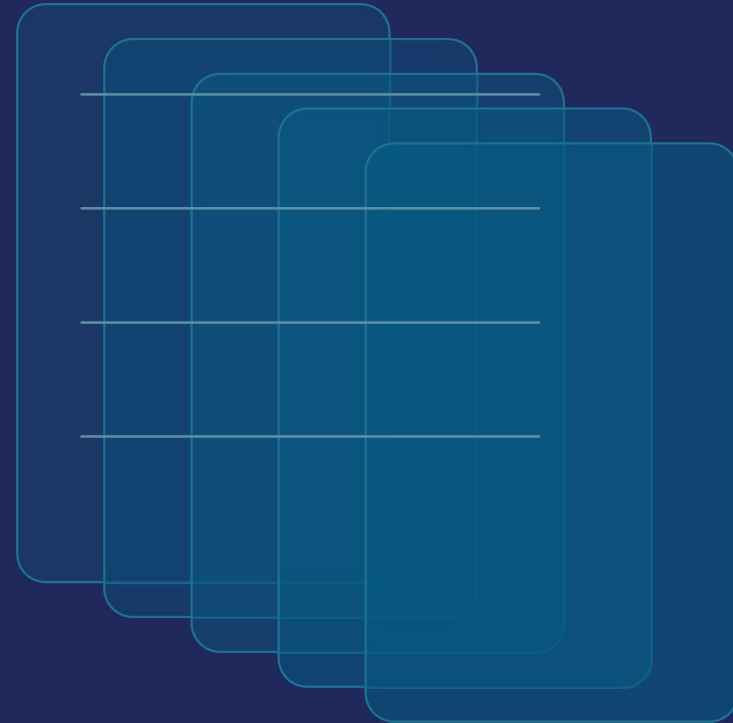
KUBERNETES TRAINING

Module 1:

Docker & Container Fundamentals

Build the mental model behind images, containers, Dockerfiles, and the Docker CLI.

2026 MODERNIZED EDITION





What we will build today

01

Why containers?

Deployment pain → consistent environments

02

Containers vs VMs

Isolation, overhead, portability

03

Docker mechanics

Architecture + image workflows

04

Operate containers

CLI + lifecycle + inspection

05

Build images

Layers + registries + Dockerfile

06

2026 context

Docker builds locally; CRI runtimes in clusters



Why containers?

WITHOUT CONTAINERS

- Install runtime itself
- Install app dependencies
- Match versions across hosts
- Coordinate the deployment script
- Repeat the same work for each app

Devs → Ops

WITH CONTAINERS

- 1 Write a Dockerfile
- 2 Build + push one image
- 3 Run the same packaged environment
- 4 Ops supplies only what is needed
- 5 Deploy the application

Result: simplified setup • portability • consistency • isolation • efficiency • resource control • scalability



Containers vs Virtual Machines

Dimension	Virtual Machines	Docker Containers
Isolation	Strong: each VM has its own OS.	Process-level: containers share the host OS kernel.
Size / overhead	Larger footprint: guest OS + virtual hardware.	Lightweight: minimal overhead; shared kernel.
Portability	Can be tied to hypervisor + guest OS configuration.	Highly portable; consistent across environments.

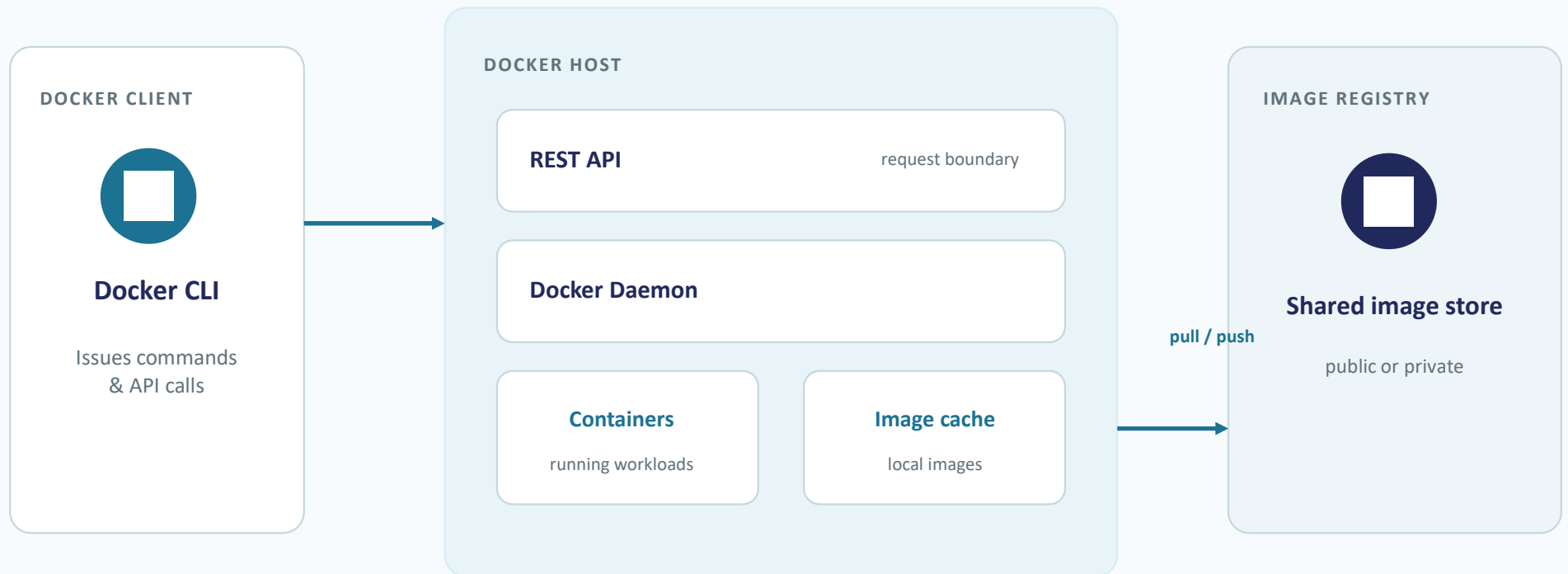
Use VMs when...

strong isolation • legacy workloads • complete-system testing

Use containers when...

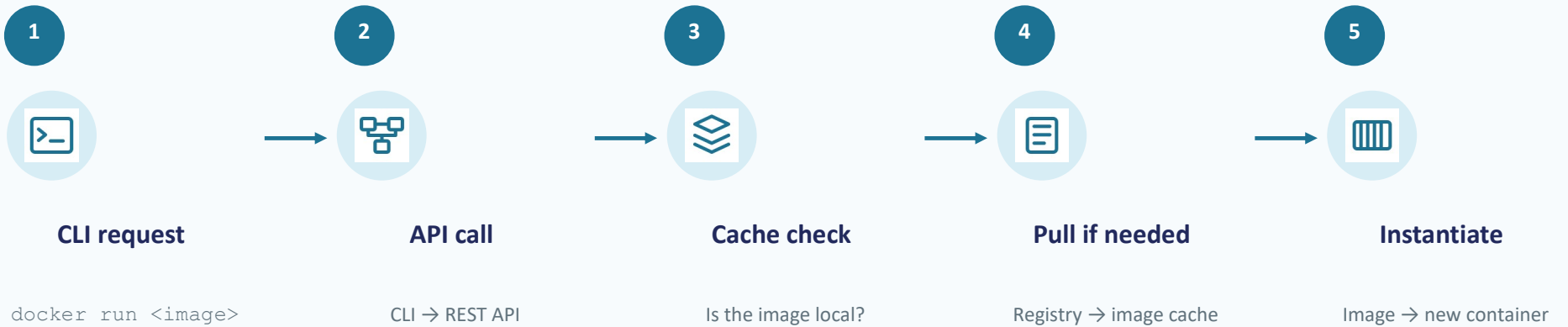
cloud-native apps • microservices • rapid scaling • portability

Docker architecture





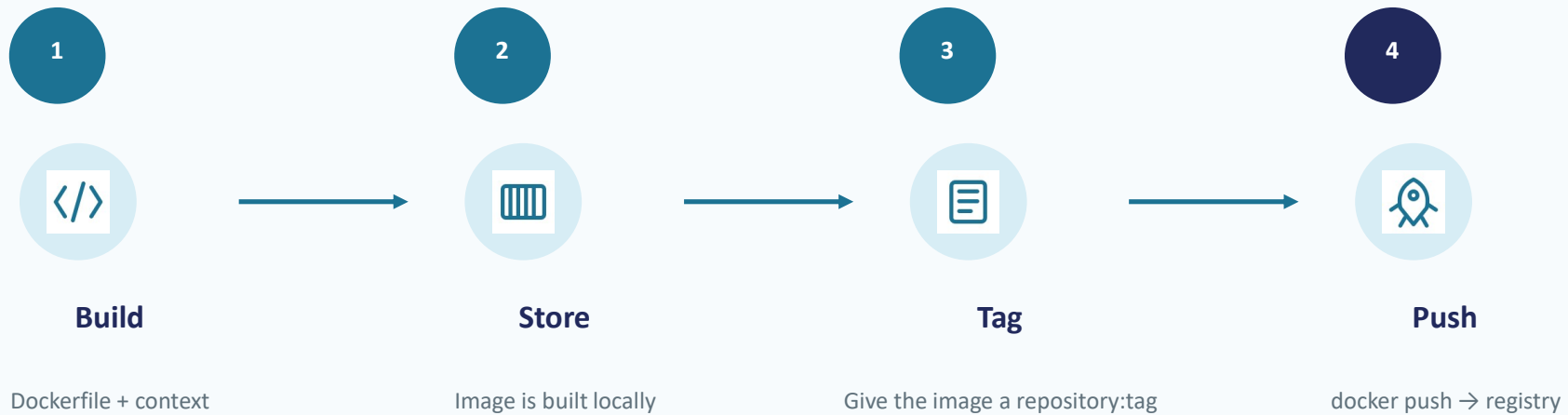
Run a container from an image



Key idea Docker only downloads the image when it is not already available in the local cache.



Build and push an image



terminal

```
docker build -t my_first_image .  
docker push my_first_image
```

Docker CLI basics

Pull

```
docker pull nginx
```

Download an image

Run

```
docker run nginx
```

Create + start

Detached

```
docker run -d nginx
```

Run in background

List

```
docker ps
```

Show running containers

Stop

```
docker stop <container_name>
```

Graceful stop

Kill

```
docker kill <container_name>
```

Force stop

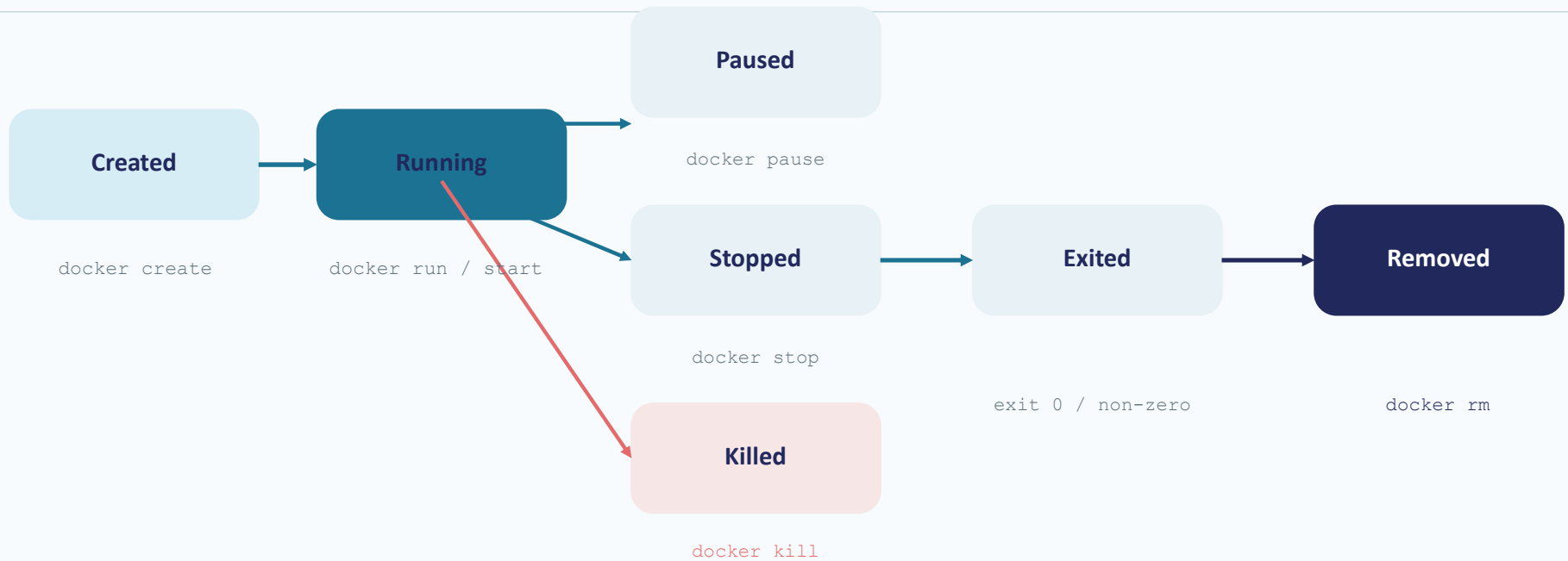
Publish

```
docker run -d -p 8080:80 --name webserver  
nginx
```

Host 8080 → container 80

Then verify: `curl http://localhost:8080`

Container lifecycle



Stopped ≠ removed • Paused keeps memory contents • Killed forces termination • Exited keeps the container until removed



Lifecycle takeaways



`docker run`

Shortcut for create + start.



`pause / unpause`

Freeze and resume without stopping the container.



`exit codes`

0 = normal completion; non-zero = error.



`stop / start`

Stopped containers still exist and can be restarted.



`rm`

Removing a container deletes it from the system.



`restart policy`

Can automatically restart failed containers.

CLI reference



`docker images`

List locally stored images.



`docker image rm <image_id>`

Remove an image from local storage.



`docker ps`

Show only running containers.



`docker ps -a`

Show running + stopped containers.



`docker ps -q`

Return only container IDs.



`docker ps --filter "name=<name>"`

Filter containers by name.



CLI reference

terminal

```
docker run -d -p 80:80 --name web_server  
nginx  
docker stop <container_name>  
docker rm <container_name>
```

- d** Detached mode — run in the background.
- p 80:80** Map host port 80 → container port 80.
- name** Give the container a predictable name.
- stop** Graceful shutdown.
- rm** Remove a stopped container and free resources.

Graceful stop helps prevent data corruption.

CLI reference

View container logs

Use `docker logs` for debugging; add `-f` to follow logs live.

Typical check: `curl http://localhost` can generate a GET request in an Nginx container.

 terminal

```
docker logs <container_name>
docker logs -f <container_name>
```

Execute inside a running container

Open a shell, inspect files, test commands, or read the content being served.

Alpine commonly uses `sh`; Bash may be available on Ubuntu-based images.

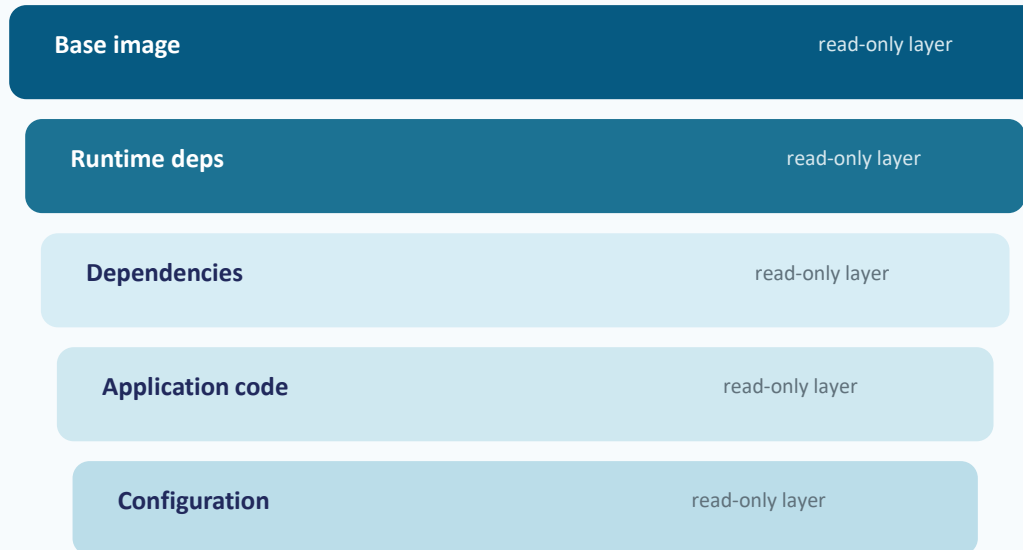
 terminal

```
docker exec -it <container_name> sh
docker exec -it <container_name> /bin/bash
```

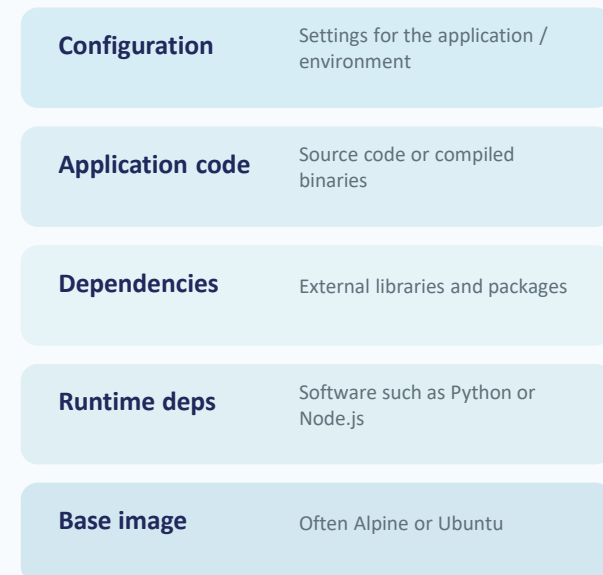


Docker images

An image is a self-contained, read-only template that packages what the application needs to run.

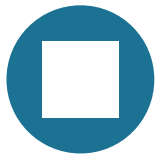


Think “snapshot”: frozen, repeatable, ready to become a container.





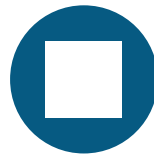
Where images come from



Docker Hub

Broad public catalog of images.

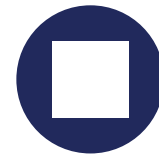
START HERE



Private registries

Secure storage for proprietary / sensitive images.

CONTROL ACCESS



Build your own

Write a Dockerfile that matches your application.

CUSTOMIZE

Container registries



Collaboration

Share images with teammates or clients.



Versioning

Track versions for rollback and updates.



Security

Private registries protect sensitive images.



Automation

Connect image build + deployment to CI/CD.

PUBLIC

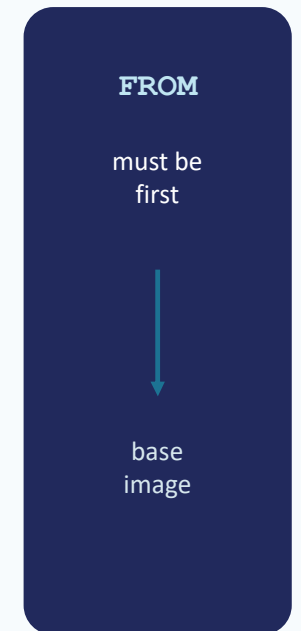
Open to everyone; Docker Hub is the prominent example in the source.

PRIVATE

Proprietary / sensitive images with granular access control.

What is a Dockerfile?

- 1 Recipe** Step-by-step instructions to build a Docker image.
- 2 Order matters** Instructions execute sequentially from top to bottom.
- 3 Layers** Each instruction contributes to the image layer history.
- 4 Result** Dependencies + environment + configuration become a repeatable image.



Dockerfile instruction reference

Instruction	Purpose	Example
FROM	Base image	<code>FROM ubuntu:latest</code>
RUN	Execute a build command	<code>RUN apt-get update && apt-get install -y curl</code>
COPY	Copy files into image	<code>COPY app /usr/src/app</code>
ADD	COPY-like + archive extraction	<code>ADD archive.tar.gz /data/</code>
WORKDIR	Set working directory	<code>WORKDIR /usr/src/app</code>
CMD	Default command at start	<code>CMD ["node", "app.js"]</code>
ENTRYPOINT	Command that always executes	<code>ENTRYPOINT ["command", "arg1"]</code>
EXPOSE	Document container port	<code>EXPOSE 8080</code>
ENV	Define environment variable	<code>ENV NODE_ENV=production</code>

Dockerfile walkthrough

The source's first custom image is intentionally tiny: define a base image, run a build-time command, then tell the container what to do when it starts.

```
Dockerfile

FROM ubuntu:latest
RUN echo "Hello from my first Docker image" >
/message.txt
CMD ["cat", "/message.txt"]
```

Read it as:

FROM choose the base image

RUN runs once, at build time

Expected output

```
Hello from my first Docker
image
```



Build & run your custom image

terminal

```
docker build -t my_first_image .  
docker images  
docker run my_first_image
```

Expected output

```
Hello from my first Docker image
```



Practice roadmap: from run to production-ready

01

Run with real options

Env vars, detached mode, verify with curl

02

Inspect & debug

inspect, stats, logs, exec

03

Persist data

Volumes, bind mounts, tmpfs

04

Connect containers

User-defined networks + DNS by name

05

Ship it properly

Multi-stage builds + .dockerignore

06

Clean up

Prune safely, without regret



Run a container the way production does

terminal

```
docker run -d \  
  --name webserver -p 8080:80 \  
  -e NGINX_HOST=learning.local nginx
```

-d

Detached mode — runs in the background, frees your terminal.

-p 8080:80

Maps host port 8080 to container port 80.

-e KEY=value

Injects an environment variable at runtime — no rebuild needed.

**--env-file
.env**

Loads many variables at once from a file.

--name

Gives the container a predictable, referenceable name.

A **-e** flag overrides the same key set by ENV in the Dockerfile.

Inspect & debug like an operator

Inspect

```
docker inspect webserver
```

Full JSON: config, mounts, network, env

Field

```
docker inspect -f '{{.State.Status}}' webserver
```

One field only — no JSON noise

Stats

```
docker stats webserver
```

Live CPU / memory / network usage

Processes

```
docker top webserver
```

List processes running inside

Logs

```
docker logs -f webserver
```

Stream logs live

Shell

```
docker exec -it webserver sh
```

Interactive shell inside

Ports

```
docker port webserver
```

Show published port mappings

Key idea: inspect before you exec — most answers already live in the metadata.



Persist data with volumes

Approach	Command	When to use it
Named volume	<code>docker run -v app-data:/var/lib/data nginx</code>	Docker manages the storage location; survives container removal.
Bind mount	<code>docker run -v \$(pwd)/html:/usr/share/nginx/html nginx</code>	Maps a host folder directly; best for local development.
tmpfs mount	<code>docker run --tmpfs /app/cache nginx</code>	In-memory only — gone when the container stops.

Named volumes when...

Docker manages the data • production workloads • portable across hosts

Bind mounts when...

local development • live-reload workflows • exact host path needed

Connect containers with a network



`docker network ls`

List all networks on this host.



`docker network create app-net`

Create a user-defined bridge network.



`docker run --network app-net --name db postgres`

Attach the db container to app-net.



`docker run --network app-net -e DB_HOST=db --name api myapi`

api reaches db by container name — automatic DNS.



`docker network inspect app-net`

See connected containers and their IPs.



`docker network rm app-net`

Remove once no containers are attached.

Ship it: multi-stage builds

Compile in a throwaway stage, then copy only the finished output into a minimal final image.

 Dockerfile

```
FROM node:20 AS build
WORKDIR /app
COPY . .
RUN npm ci && npm run build

FROM nginx:alpine
COPY --from=build /app/dist /usr/share/nginx/html
```

Read it as:

Stage 1 install deps & compile

Stage 2 copy only the build output

Result

A small final image — no compilers, no dev deps, no build cache



Keep secrets and bloat out: .dockerignore

```
.dockerignore  
  
node_modules  
.git  
*.md
```

Result

A smaller build context, faster builds, and no secrets copied into the image by accident.

Clean up like a pro



`docker system df`

See what is using disk space.



`docker container prune`

Remove all stopped containers.



`docker image prune -a`

Remove unused images.



`docker volume prune`

Remove unused volumes.



`docker network prune`

Remove unused networks.



`docker system prune -a --volumes`

Remove everything unused at once — use with care.



Docker in 2026 — What changed

WHAT YOU BUILD HERE STILL APPLIES

- 1 Docker Engine remains useful for local development and image builds.
- 2 Docker-produced OCI images remain portable across compatible runtimes.
- 3 The Kubernetes cluster runtime is a separate concern from the image build tool.

CLUSTER RUNTIME

Kubernetes removed dockershim in v1.24.

Today: use a CRI-compatible runtime such as containerd or CRI-O.

Forward-looking note

Kubernetes has been dropping support for older containerd 1.x releases across recent versions — containerd 2.0+ is the direction of travel. Check your target cluster's supported runtime versions before upgrading.

Module 2 shows what runs your containers in a cluster.

Verify current runtime support against the Kubernetes release notes for your cluster version

Runtime details evolve fast — always confirm against current docs



Module recap



Why containers reduce deployment friction



Container vs VM trade-offs



Docker client → API → daemon → cache / registry



Pull, run, build, tag, push



Container states and lifecycle commands



Images as read-only layered templates



Public vs private registries



Dockerfile instructions + ordering



Build and run a custom image

If you can tell the story end-to-end, you are ready for Module 2.

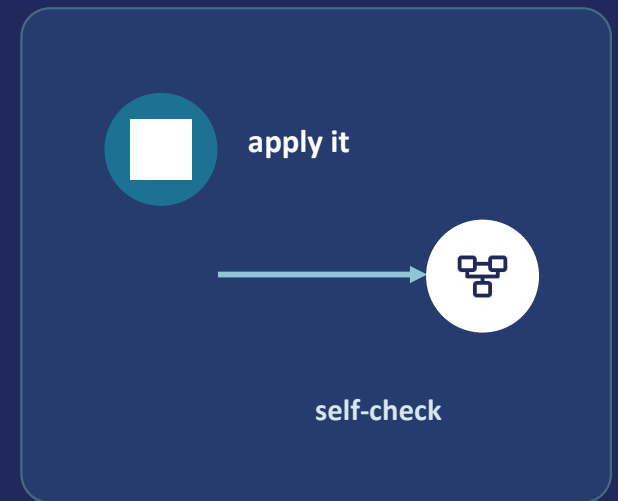


KNOWLEDGE CHECK

9 questions

Before you move on

Scenario-based questions covering containers, Docker architecture, the CLI, and the Dockerfile. Answer key on the last slide.





Question 1 of 9

scenario

Your team needs to run a stateless web app on 50 servers with guaranteed identical behavior regardless of host OS patches. What is the PRIMARY reason containers solve this better than manually configuring each VM?

A

Containers package the app with its exact dependencies, so behavior doesn't depend on what's installed on the host

B

Containers use less disk space than VMs

C

Containers always run faster than VMs

D

Containers cannot be affected by host OS updates at all

Full explanations for every question are in the Answer Key on the last slide.



Question 2 of 9

scenario

A teammate says: “Let's use a VM instead of a container here — we need strong OS-level isolation for a legacy compliance workload.” Is this a reasonable call?

A

No — containers always provide stronger isolation than VMs

B

No — VMs and containers provide identical isolation guarantees

C

Yes — VMs isolate at the OS/kernel level, which containers (sharing the host kernel) don't provide

D

Yes, but only because VMs are cheaper to run

Full explanations for every question are in the Answer Key on the last slide.



Question 3 of 9

scenario

You need to stop a container that is actively writing to a database file, and want to minimize the risk of data corruption. Which command should you use?

- A `docker kill`
- B `docker rm -f`
- C `docker pause`
- D `docker stop`

Full explanations for every question are in the Answer Key on the last slide.



Question 4 of 9

scenario

You build an image with only a RUN instruction (no CMD or ENTRYPOINT). You run it with `docker run myapp` and it exits immediately with no output. What's the most likely explanation?

A

RUN only executes during the build; without CMD/ENTRYPOINT there's nothing to run at container start

B

The base image is corrupted

C

Docker requires at least two RUN instructions per Dockerfile

D

The container needs the `-d` flag to produce output

Full explanations for every question are in the Answer Key on the last slide.



Question 5 of 9

scenario

A teammate says: “Always put COPY . . as early as possible in the Dockerfile for speed.” Why is this generally bad advice for build caching?

- A COPY must always come after CMD
- B Docker ignores instruction order entirely
- C COPY instructions don't create layers
- D Placing frequently-changing files (like source code) early invalidates the cache for every layer after it — even slow ones like dependency installs

Full explanations for every question are in the Answer Key on the last slide.



Question 6 of 9

scenario

Your deployment pipeline needs to guarantee the exact same image bytes reach staging and production. Which practice most directly supports this?

- A Always use the :latest tag
- B Rebuild the image separately in each environment
- C Tag images with an immutable identifier — like a git commit SHA — instead of :latest
- D Use a different base image per environment

Full explanations for every question are in the Answer Key on the last slide.



Question 7 of 9

scenario

Two containers — an API and a database — need to reach each other by name (e.g. the API connects to db:5432) without hardcoding IP addresses. What's the correct approach?

- A Manually edit `/etc/hosts` inside both containers
- B Create a user-defined network and attach both containers to it
- C Use `--link`, which is the modern recommended approach
- D Use the default bridge network — name resolution works automatically there

Full explanations for every question are in the Answer Key on the last slide.



Question 8 of 9

scenario

A container writes important data to `/var/lib/data` inside itself. You remove the container with `docker rm`, and the data is gone. What should have been done differently?

A

Mount a volume or bind mount at `/var/lib/data` before writing the data

B

Nothing could have prevented this — container data is always ephemeral

C

Use `docker pause` instead of `docker rm`

D

Increase the container's memory limit

Full explanations for every question are in the Answer Key on the last slide.



Question 9 of 9

scenario

Your CI pipeline builds a Node.js app image that ends up 1.4GB, containing the full npm toolchain and node_modules cache used only during the build. What's the standard fix to ship a smaller, safer production image?

- A Add more RUN apt-get clean instructions
- B Use a multi-stage build — compile in one stage, copy only the build output into a minimal final stage
- C Switch to the :latest tag
- D Remove the Dockerfile after building

Full explanations for every question are in the Answer Key on the last slide.



Answer key

Q#	Correct	Why
Q1	A	Deps travel with the app – host installs don't matter
Q2	C	VMs isolate at the kernel; containers share the host kernel
Q3	D	stop sends SIGTERM (graceful); kill/rm -f are immediate
Q4	A	RUN runs at build time only – CMD is needed to run anything
Q5	D	Early COPY of changing files busts the cache for every layer after
Q6	C	An immutable tag (e.g. a commit SHA) guarantees identical bytes
Q7	B	User-defined networks give containers DNS by name
Q8	A	No volume = no persistence; data dies with the container
Q9	B	Multi-stage ships only the compiled output, not the build tools



MODULE 1 COMPLETE

Next: Module 2

Kubernetes Architecture

You now know what a container is, where its image comes from, how Docker creates and manages it, and how to package your own image.

